Power Cloud
CONSULTING

Script Reader — Tool Guide

Read and evaluate real SuiteScript source straight through the NetSuite AI Connector — no downloads, no guesswork, no reliance on memory of why a script was built.

- NetSuite AI Connector · MCP
- 4 read-only tools
- SuiteCloud (ACP) · 2026.1

OVERVIEW

Why this exists

The NetSuite MCP server is excellent at returning records, metadata, and query results — but it cannot open the actual body of a script file. The built-in SuiteQL tools can tell you a script's *name, type, and file size*, yet never a single line of its *code*, because the File Cabinet's content simply isn't exposed to SuiteQL.

The **Power Cloud Script Reader** closes that gap. It is a custom toolset deployed into a NetSuite account that surfaces four read-only tools inside the same MCP connection you already use. With them, an assistant can open an existing customization, understand exactly what it does, and flag mistakes or inefficiencies — all in conversation.

THE TOOLKIT

Read Script Source

`readScriptSource`

Open the real code of any SuiteScript by id, internal id, name, file id, or path.

List Scripts

`listScripts`

Discover what scripts exist — filter by name or type before reading one.

Read File

`readFile`

Open any File Cabinet text file — XML, JSON, CSV, templates — not just scripts.

Get Script Deployments

`getScriptDeployments`

See how and when a script actually runs — status, context, log level.

GETTING CONNECTED

How you reach the tools

The tools live in the account's **NetSuite AI Connector** (MCP) server and appear automatically alongside the built-ins such as `ns_getRecord` and `ns_runCustomSuiteQL` once two conditions are met:

- The Script Reader toolset is **deployed** into the account you're connecting to.
- The role used for the MCP connection holds **File Cabinet (View)** — this is the permission gate that makes the tools visible and lets them read.

After connecting, you simply ask in plain language. The assistant selects the right tool — you don't call anything by hand.

> **If the tools don't appear after connecting**
>
> That's the signal the connecting role is missing **File Cabinet (View)**. Grant it on the role and reconnect — tool lists are cached at connection time.

TOOL REFERENCE

The four tools in detail

Read Script Source

`readScriptSource`

READ-ONLY

Returns the actual source of a SuiteScript file as plain text. Provide any one of a script identifier, a File Cabinet file id, or a file path. Large or minified files can be paged with a line range so they stay within the assistant's limits.

PARAMETER	TYPE	NOTES
`scriptId` OPT	string	Custom id (e.g. `customscript_pc_foo`), numeric internal id as a string, or the exact script name.
`fileId` OPT	number	File Cabinet internal id of the .js file. Takes precedence over the others.
`filePath` OPT	string	Full path, e.g. `/SuiteScripts/PowerCloud/foo.js`.
`startLine` OPT	number	1-based first line to return. Default `1`.
`maxLines` OPT	number	Max lines to return. Default `2000`. Pair with `startLine` to page.

Provide at least one of `scriptId`, `fileId`, or `filePath`. Resolution order: `fileId` → `filePath` → `scriptId`.

ASK IT LIKE THIS

> *"Read the source of customscript_cl_validate_email and tell me what it does."*

RETURNS


```
{
  "source": "/**\n * @NApiVersion 2.1 ...",
  "fileName": "Client_validate_email.js",
  "fileId": 650,
  "scriptName": "CL Validate Email",
  "scriptType": "CLIENT",
  "totalLines": 150, "returnedLines": 150,
  "startLine": 1, "truncated": false, "error": ""
}
```

List Scripts

`listScripts`

READ-ONLY

Lists script records with metadata only — no source — so you can discover what exists before opening one. Results are returned as a JSON-encoded array in `scriptsJson` and capped at 200 rows (`capped: true` when more exist).

PARAMETER	TYPE	NOTES
`nameFilter` OPT	string	Case-insensitive substring matched against script name or custom id.
`scriptType` OPT	string	e.g. `USEREVENT`, `CLIENT`, `SCHEDULED`, `MAPREDUCE`, `SUITELET`, `RESTLET`, `WORKFLOWACTION`, `MASSUPDATE`, `PORTLET`.
`includeInactive` OPT	boolean	Include inactive scripts. Default `false`.

ASK IT LIKE THIS

> *"List the Map/Reduce scripts with "commit" in the name."*

RETURNS (EACH ROW INSIDE SCRIPTSJSON)

```
{ "scriptName":"MRS_ScheduleCommitmentMovement",
  "scriptId":"customscript_...", "scriptType":"MAPREDUCE",
  "fileId":1099, "fileName":"MRS_ScheduleCommitmentMovement.js",
  "isInactive":false }
```

Read File

`readFile`

READ-ONLY

Opens any *text* file in the File Cabinet — XML, JSON, CSV, FreeMarker, HTML, plain text — not just scripts. Binary files (images, PDFs, archives) are refused cleanly with `isBinary: true` rather than returned as unreadable base64. Supports the same line-range paging.

PARAMETER	TYPE	NOTES
`fileId` OPT	number	File Cabinet internal id. Takes precedence over `filePath`.
`filePath` OPT	string	Full path, e.g. `/Templates/config.json`.
`startLine` / `maxLines` OPT	number	Same paging as Read Script Source.

ASK IT LIKE THIS

> *"Open /SuiteScripts/PowerCloud/scriptreader/scriptreader_schema.json and show the first 20 lines."*

RETURNS


```
{ "contents":"{ \"tools\": [ ...", "fileName":"scriptreader_schema.json",
  "fileId":3127, "fileType":"JSON", "folder":"709",
  "totalLines":190, "returnedLines":20, "truncated":true,
  "isBinary":false, "error":"" }
```

Get Script Deployments

`getScriptDeployments`

READ-ONLY

Returns the deployment records for a script — how and when it actually runs. Useful context before judging a script: a Map/Reduce on a schedule is read very differently from a User Event on a transaction. Returned as a JSON-encoded array in `deploymentsJson`.

PARAMETER	TYPE	NOTES
`scriptId` OPT	string	Custom id, numeric internal id as a string, or exact name.
`scriptInternalId` OPT	number	Numeric internal id of the script record.

ASK IT LIKE THIS

> *"Show the deployments for script 458 — is it scheduled?"*

RETURNS (EACH ROW INSIDE DEPLOYMENTSJSON)


```
{ "deploymentId":"customdeploy1", "scriptName":"MRS_ScheduleCommitmentMovement",
  "status":"Scheduled", "isDeployed":true,
  "executionContext":"", "recordType":"", "logLevel":"DEBUG" }
```


Why arrays come back as JSON strings

`listScripts` and `getScriptDeployments` return their rows inside a single JSON-encoded string (`scriptsJson` / `deploymentsJson`). This follows NetSuite's custom-tool guidance to stringify structured output, and the assistant parses it for you automatically.

IN PRACTICE

Common workflows

Evaluate a script for errors & efficiency

The headline use case. Combine the source with its deployment context for a grounded review.

> *"Evaluate script 458 for errors and efficiency."*

The assistant reads the source, checks how it's deployed, and reports concrete findings — dead code paths, swallowed errors, governance waste — rather than guessing from a name.

Audit a customization footprint

> *"List every User Event script, then summarise which transactions they touch."*

Understand a config or template

> *"Read the FreeMarker template at /Templates/invoice_pc.ftl and explain the conditional logic."*

> **Working with large or minified files**
>
> When `truncated: true` comes back, ask for the next range — e.g. *"continue from line 2000"*. The default window is 2,000 lines; files over 10 MB are reported rather than loaded, to protect both NetSuite governance and the assistant's context.

GOOD TO KNOW

Behaviour & limits

- **Strictly read-only.** Every tool only reads — none can create, edit, or delete anything in the account.
- **Source comes back as clean text.** JavaScript files are returned verbatim — comments, formatting, and all — never base64.
- **Binary files are refused,** not dumped. `readFile` flags `isBinary: true` for images, PDFs, and archives.
- **Paging keeps things safe.** Use `startLine` / `maxLines` to walk through big files; the `truncated` flag tells you when there's more.
- **Errors are reported, never thrown.** Each tool returns a human-readable `error` field instead of failing the call — an unknown id yields a clear message, not a crash.
- **Visibility follows permissions.** The connecting role only sees scripts and files it is allowed to view.

FOR ADMINISTRATORS

Deploying into another account

The Script Reader ships as a SuiteCloud **Account Customization Project (ACP)**, so it installs straight into any account you administer — no SDN publisher id required.

1. **Authenticate with a deploy role.** Run `npx suitecloud account:setup` and sign in with an *Administrator* (or SuiteApp-Deployment-capable) role — not the locked-down MCP role.

2 **Deploy.** Run `npx suitecloud project:deploy`. It uploads the script and schema and registers the toolset object.

3 **Grant the MCP role File Cabinet (View).** The role used for the AI Connector connection needs this to see and run the tools — keep that role otherwise minimal.

4 **Connect & verify.** Connect the AI Connector, confirm the toolset under *Customization → Scripting → Custom Tools*, and test with a prompt.

> **Two roles, two jobs**
>
> Deployment uses an admin/SDF role; the live MCP connection uses a minimal, non-Administrator role. The AI Connector does not support Administrator roles — keep them separate by design.